PATAN MULTIPLE CAMPUS

PATAN DHOKA, LALITPUR

(Department of Computer Science and Information Technology)

ADVANCED JAVA PROGRAMMING

LAB REPORT

BSc. CSIT - 7th Semester

Submitted By:

Name: **Samir Paudel**
Program: BSc. CSIT
Semester: 7th Semester
Section: D
Roll No: 114/079
Symbol No: **79010103**

Submitted To:

Department of Computer Science and Information Technology
Patan Multiple Campus
Patan Dhoka, Lalitpur
Tribhuvan University

INDEX

S.N.	Experiment / Lab Task	Date	Signature
1	Box Class	July 2, 2026	
2	Matrix and Jagged Array	July 2, 2026	
3	Static Variables and Final Keyword	July 2, 2026	
4	Inheritance - Student Hierarchy	July 2, 2026	
5	Super Keyword and Multilevel Inheritance	July 2, 2026	
6	Method Overriding and Polymorphism	July 2, 2026	
7	Nested Classes	July 2, 2026	
8	Interface Implementation	July 2, 2026	
9	Exception Handling	July 2, 2026	
10	Threading	July 2, 2026	
11	File I/O and Serialization	July 2, 2026	
12	Swing - Colored Buttons	July 2, 2026	
13	Simple GUI - Text Field	July 2, 2026	
14	Text Area with File Operations	July 2, 2026	
15	Menu Bar	July 2, 2026	
16	JTextField with KeyListener	July 2, 2026	
17	JCheckBox and JRadioButtons	July 2, 2026	
18	JDBC - MOVIE Table Operations	July 2, 2026	
19	Scrollable ResultSet	July 2, 2026	
20	Updatable ResultSet	July 2, 2026	
21	TCP Sockets - One-Way Chat	July 2, 2026	
22	UDP Echo Server and Client	July 2, 2026	
23	URL Object Handling	July 2, 2026	
24	Email Using JavaMail API	July 2, 2026	

Lab 1: Box Class

Title: Create a class named ”Box” which includes the variables: length, breadth, and height. Add a method to print the values of these variables. Add another method that prints the volume of box. Now create two objects of ”Box” class inside main method and initialize the variables for both objects then call above methods using each object.

Theory

Classes and objects form the foundation of OOP. A class is a blueprint, and an object is an instance. Instance variables belong to each object, and methods define behavior. The `new` keyword allocates memory and returns a reference.

Code

```java
public class Lab1_BoxClass {
    static class Box {
        double length;
        double breadth;
        double height;
        void printValues() {
            System.out.println("Length: " + length);
            System.out.println("Breadth: " + breadth);
            System.out.println("Height: " + height);
        }
        void printVolume() {
            double volume = length * breadth * height;
            System.out.println("Volume of Box: " +
                volume);
        }
    }
    public static void main(String[] args) {
        Box box1 = new Box();
        box1.length = 5.0;
        box1.breadth = 4.0;
        box1.height = 3.0;
        System.out.println("--- Box 1 ---");
        box1.printValues();
        box1.printVolume();
        Box box2 = new Box();
        box2.length = 10.0;
        box2.breadth = 8.0;
        box2.height = 6.0;
        System.out.println("\n--- Box 2 ---");
        box2.printValues();
        box2.printVolume();
        System.out.println("\nLab No.: 1");
        System.out.println("Name: Samir Paudel");
        System.out.println("Roll No./Section: 114-079/
            D");
    }
}
```

Output

```
[sam@arch labs]$  javac Lab1_BoxClass.java && java Lab1_BoxClass
--- Box 1 ---
Length: 5.0
Breadth: 4.0
Height: 3.0
Volume of Box: 60.0

--- Box 2 ---
Length: 10.0
Breadth: 8.0
Height: 6.0
Volume of Box: 480.0

============================
Lab No.: 1
Name: Samir Paudel
Roll No./Section: 114-079/D
============================
```

Lab 2: Matrix and Jagged Array

Title: WAP to take the elements of a 3x3 matrix from user and display the matrix and all diagonal elements. Also WAP to demonstrate the concept of jagged array.

Theory

Multidimensional Arrays: A 2D array has rows and columns. The main diagonal runs from top-left to bottom-right (indices i,i). The anti-diagonal runs from top-right to bottom-left (indices i, n-1-i).

Jagged Array: An array of arrays where each row can have different lengths. Memory efficient for irregular data structures.

Code

```java
import java.util.Scanner;
public class Lab2_MatrixAndJaggedArray {
    static void displayMatrixAndDiagonals() {
        Scanner sc = new Scanner(System.in);
        int[][] matrix = new int[3][3];
        System.out.println("Enter elements for 3x3 matrix:");
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                System.out.print("Element [" + i + "][" + j + "]: ");
                matrix[i][j] = sc.nextInt();
            }
        }
        System.out.println("\nMatrix:");
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                System.out.print(matrix[i][j] + " ");
            }
            System.out.println();
        }
        System.out.println("\nMain Diagonal Elements:");
        for (int i = 0; i < 3; i++) {
            System.out.print(matrix[i][i] + " ");
        }
        System.out.println("\n\nAnti-Diagonal Elements:");
        for (int i = 0; i < 3; i++) {
            System.out.print(matrix[i][2 - i] + " ");
        }
        System.out.println();
    }
    static void demonstrateJaggedArray() {
        System.out.println("\n--- Task B: Jagged Array ---");
        int[][] jaggedArray = {
            {1, 2, 3},
            {4, 5},
            {6, 7, 8, 9},
            {10}
        };
        System.out.println("Jagged Array elements:");
        for (int i = 0; i < jaggedArray.length; i++) {
            System.out.print("Row " + i + ": ");
            for (int j = 0; j < jaggedArray[i].length; j++) {
                System.out.print(jaggedArray[i][j] + " ");
            }
            System.out.println();
        }
    }
    public static void main(String[] args) {
        displayMatrixAndDiagonals();
        demonstrateJaggedArray();
        System.out.println("\nLab No.: 2");
        System.out.println("Name: Samir Paudel");
        System.out.println("Roll No./Section: 114-079/D");
    }
}
```

Output

```
--- Task A: Matrix 3x3 and Diagonal Elements ---
Enter elements for 3x3 matrix:
Element [0][0]: 1
Element [0][1]: 2
Element [0][2]: 3
Element [1][0]: 4
Element [1][1]: 5
Element [1][2]: 6
Element [2][0]: 7
Element [2][1]: 8
Element [2][2]: 9

Matrix:
1 2 3
4 5 6
7 8 9

Main Diagonal Elements:
1 5 9

Anti-Diagonal Elements:
3 5 7

--- Task B: Jagged Array ---
Jagged Array elements:
Row 0: 1 2 3
Row 1: 4 5
Row 2: 6 7 8 9
Row 3: 10

============================
Lab No.: 2
Name: Samir Paudel
Roll No./Section: 114-079/D
============================
[sam@arch labs]$
```

Lab 3: Static and Final Keywords

Title: WAP in Java to demonstrate: a) Static variable, static method, static block b) Final keyword (variable/method/class)

Theory

Static Variables: Shared across all instances. Initialized once when class loads. Accessed via class name.

Static Methods: Can be called without object. Cannot access non-static members directly.

Static Block: Executed once when class is loaded into memory. Used for static initialization.

Final Keyword: Applied to variables (constant, cannot reassign), methods (cannot override), classes (cannot extend).

Code

```java
public class Lab3_StaticAndFinal {
    static int staticCount = 0;
    final int finalValue = 100;
    static void displayStaticCount() {
        staticCount++;
        System.out.println("Static Count: " +
            staticCount);
    }
    static {
        System.out.println("--- Static Block Executed
            ---");
        staticCount = 10;
        System.out.println("Initial static count set
            to: " + staticCount);
    }
    static class Parent {
        final void finalMethod() {
            System.out.println("Final method - cannot
                override");
        }
    }
    static class Child extends Parent {
        // void finalMethod() {} // would cause
            compile error
    }
    final class FinalClass {
        void display() {
            System.out.println("Final class - cannot
                extend");
        }
    }
    // class SubClass extends FinalClass {} // would
        cause compile error
    static void taskStaticDemo() {
        System.out.println("\n--- Task A: Static ---")
            ;
        displayStaticCount();
        displayStaticCount();
        System.out.println("Final static count: " +
            staticCount);
    }
    static void taskFinalDemo() {
        System.out.println("\n--- Task B: Final
            Keyword ---");
        final int x = 50;
        System.out.println("Final variable x: " + x);
        final String finalString = "This is final";
        System.out.println("Final string: " +
            finalString);
        Child child = new Child();
        child.finalMethod();
        FinalClass fc = new FinalClass();
        fc.display();
        System.out.println("Final method and class
            demonstrated");
    }
    public static void main(String[] args) {
        taskStaticDemo();
        taskFinalDemo();
        System.out.println("\nLab No.: 3");
        System.out.println("Name: Samir Paudel");
        System.out.println("Roll No./Section: 114-079/
            D");
    }
}
```

Output

```
[sam@arch labs]$  javac Lab3_StaticAndFinal.java && java Lab3_StaticAndFinal
--- Static Block Executed ---
Initial static count set to: 10

--- Task A: Static Variable, Method, and Block ---
Static Count: 11
Static Count: 12
Final static count: 12

--- Task B: Final Keyword ---
Final variable x: 50
Final string: This is final

============================
Lab No.: 3
Name: Samir Paudel
Roll No./Section: 114-079/D
============================
```

Lab 4: Inheritance - Student Hierarchy

Title: Create a class Student with instance variable roll_no and two methods to read and display the roll no. Then, create another class Test that inherits class Student, consisting of its own instance variables to hold the marks of two subjects and also methods to read and display the marks. Finally, create another class Result which inherits class Test. It also has its own instance variable total that holds the total of two marks scored by the student. Similarly, it has methods to calculate and display the total. Create some instances of above classes and demonstrate inheritance.

Theory

Inheritance: Mechanism where a class acquires properties and methods of another class using `extends`. Promotes code reusability.

Types: Single (one parent), Multilevel (chain), Hierarchical (multiple children).

Constructor Chaining: When a subclass object is created, parent class constructor is called first (automatically or via `super()`).

Code

```java
public class Lab4_StudentHierarchy {
    static class Student {
        protected int roll_no;
        void readRollNo(int roll) {
            this.roll_no = roll;
        }
        void displayRollNo() {
            System.out.println("Roll No: " + roll_no);
        }
    }
    static class Test extends Student {
        protected int marks1;
        protected int marks2;
        void readMarks(int m1, int m2) {
            this.marks1 = m1;
            this.marks2 = m2;
        }
        void displayMarks() {
            System.out.println("Marks 1: " + marks1);
            System.out.println("Marks 2: " + marks2);
        }
    }
    static class Result extends Test {
        int total;
        void calculateTotal() {
            this.total = marks1 + marks2;
        }
        void displayTotal() {
            System.out.println("Total Marks: " + total
                );
        }
    }
    public static void main(String[] args) {
        Result result = new Result();
        result.readRollNo(114);
        System.out.println("Student Information:");
        result.displayRollNo();
        result.readMarks(85, 90);
        result.displayMarks();
        result.calculateTotal();
        result.displayTotal();
        System.out.println("\nLab No.: 4");
        System.out.println("Name: Samir Paudel");
        System.out.println("Roll No./Section: 114-079/
            D");
    }
}
```

Output

```
[sam@arch labs]$  javac Lab4_StudentHierarchy.java && java Lab4_StudentHierarchy
--- Inheritance Demonstration ---

Student Information:
Roll No: 114
Marks 1: 85
Marks 2: 90
Total Marks: 175

============================
Lab No.: 4
Name: Samir Paudel
Roll No./Section: 114-079/D
============================
```

Lab 5: Super Keyword and Multilevel Inheritance

Title: WAP to demonstrate: a) The use of super keyword to access super class constructor b) The use of super keyword to overcome name hiding (to access parent class members from child class) c) Multilevel inheritance in Java

Theory

Super Keyword: References the immediate parent class. Used to:

- Call parent constructor: `super(args)`
- Access parent members hidden by child: `super.member`
- Call parent method: `super.method()`

Multilevel Inheritance: A chain where GrandChild → Child → Parent. Each subclass can access all ancestors' members.

Code

```java
public class Lab5_SuperKeyword {
    static class GrandParent {
        void display() {
            System.out.println("GrandParent display");
        }
    }
    static class Parent extends GrandParent {
        String name = "Parent";
        Parent() {
            System.out.println("Parent Constructor");
        }
        Parent(String name) {
            this.name = name;
            System.out.println("Parent Constructor: "
                + name);
        }
        void display() {
            System.out.println("Parent display - Name:
                 " + name);
        }
    }
    static class Child extends Parent {
        String name = "Child";
        Child() {
            super("Parent from Child");
            System.out.println("Child Constructor");
        }
        void callParentConstructor() {
            System.out.println("\nTask A: Super
                constructor");
            System.out.println("Child name: " + name);
            System.out.println("Parent name via super:
                 " + super.name);
        }
        void overriddenDisplay() {
            System.out.println("\nTask B: Name hiding"
                );
            System.out.println("Child display - " +
                name);
            super.display();
        }
    }
    static class GrandChild extends Child {
        void demonstrateMultilevel() {
            System.out.println("\nTask C: Multilevel
                Inheritance");
            display();
        }
    }
    public static void main(String[] args) {
        Child child = new Child();
        child.callParentConstructor();
        child.overriddenDisplay();
        System.out.println("\n--- Multilevel ---");
        GrandChild grandChild = new GrandChild();
        grandChild.demonstrateMultilevel();
        System.out.println("\nLab No.: 5");
        System.out.println("Name: Samir Paudel");
        System.out.println("Roll No./Section: 114-079/
            D");
    }
}
```

Output

```
[sam@arch labs]$  javac Lab5_SuperKeyword.java && java Lab5_SuperKeyword
--- Super Keyword and Multilevel Inheritance ---

Parent Constructor with name: Parent from Child
Child Constructor

Task A: Accessing super class constructor
Child name: Child
Parent name accessed via super: Parent from Child

Task B: Overcoming name hiding
Child display - Child
Parent display method - Name: Parent from Child

--- Multilevel Inheritance ---
Parent Constructor with name: Parent from Child
Child Constructor

Task C: Multilevel Inheritance
GrandChild accessing Parent methods through Child
Child display method - Name: Child

============================
Lab No.: 5
Name: Samir Paudel
Roll No./Section: 114-079/D
============================
```

Lab 6: Method Overriding and Polymorphism

Title: WAP to: a) Show method overriding in Java b) Illustrate runtime polymorphism in Java (Hint: Area of Figures - Rectangle, Triangle)

Theory

Method Overriding: Subclass provides a specific implementation of a method from parent class. Method signature must be identical. The `@Override` annotation is optional but recommended.

Runtime Polymorphism: The method to call is determined at runtime based on the actual object type, not the reference type. Achieved via method overriding.

Code

```java
public class Lab6_MethodOverriding {
    static class Shape {
        void draw() {
            System.out.println("Drawing a shape");
        }
    }
    static class Circle extends Shape {
        @Override
        void draw() {
            System.out.println("Drawing a Circle");
        }
        double getArea(double radius) {
            return Math.PI * radius * radius;
        }
    }
    static class Rectangle extends Shape {
        @Override
        void draw() {
            System.out.println("Drawing a Rectangle");
        }
        double getArea(double length, double width) {
            return length * width;
        }
    }
    static class Triangle extends Shape {
        @Override
        void draw() {
            System.out.println("Drawing a Triangle");
        }
        double getArea(double base, double height) {
            return 0.5 * base * height;
        }
    }
    static void demonstratePolymorphism() {
        Shape shape1 = new Circle();
        Shape shape2 = new Rectangle();
        Shape shape3 = new Triangle();
        shape1.draw();
        shape2.draw();
        shape3.draw();
        Circle circle = (Circle) shape1;
        System.out.println("Circle area (r=5): " +
            circle.getArea(5));
        Rectangle rect = (Rectangle) shape2;
        System.out.println("Rectangle area (10x5): " +
            rect.getArea(10, 5));
        Triangle tri = (Triangle) shape3;
        System.out.println("Triangle area (b=10, h=6): " +
            tri.getArea(10, 6));
    }
    public static void main(String[] args) {
        Circle c = new Circle();
        Rectangle r = new Rectangle();
        Triangle t = new Triangle();
        c.draw();
        r.draw();
        t.draw();
        demonstratePolymorphism();
        System.out.println("\nLab No.: 6");
        System.out.println("Name: Samir Paudel");
        System.out.println("Roll No./Section: 114-079/D");
    }
}
```

Output

```
[sam@arch labs]$  javac Lab6_MethodOverriding.java && java Lab6_MethodOverriding
--- Method Overriding and Polymorphism ---

Task A: Method Overriding
Drawing a Circle
Drawing a Rectangle
Drawing a Triangle

--- Task B: Runtime Polymorphism (Area of Figures) ---

Method Overriding:
Drawing a Circle
Drawing a Rectangle
Drawing a Triangle

Calculating Areas using Runtime Polymorphism:
Circle area (radius=5): 78.53981633974483
Rectangle area (length=10, width=5): 50.0
Triangle area (base=10, height=6): 30.0

============================
Lab No.: 6
Name: Samir Paudel
Roll No./Section: 114-079/D
```

Lab 7: Nested Classes

Title: WAP to demonstrate: a) Static nested class b) Non-static nested class (inner class) c) Local inner class

Theory

Static Nested Class: Declared with `static` inside another class. Can be instantiated without outer class instance. Can access only static members of outer class.

Inner Class (Non-static): Requires outer class instance. Can access all members (including private) of outer class.

Local Inner Class: Defined inside a method. Scope is limited to that method. Can access `final` or effectively-final local variables.

Code

```java
public class Lab7_NestedClasses {
    static class StaticNestedClass {
        static int staticVar = 10;
        static void staticMethod() {
            System.out.println("Static Nested - Static Method");
            System.out.println("Static Variable: " + staticVar);
        }
        void instanceMethod() {
            System.out.println("Static Nested - Instance Method");
        }
    }
    class InnerClass {
        int innerVar = 20;
        void innerMethod() {
            System.out.println("Inner Class - Instance Method");
            System.out.println("Inner Variable: " + innerVar);
        }
    }
    void demonstrateLocalInnerClass() {
        final int localVar = 30;
        class LocalInnerClass {
            void display() {
                System.out.println("Local Inner - Method");
                System.out.println("Local Variable: " + localVar);
            }
        }
        LocalInnerClass local = new LocalInnerClass();
        local.display();
    }
    public static void main(String[] args) {
        System.out.println("Task A: Static Nested Class");
        StaticNestedClass.staticMethod();
        StaticNestedClass nested = new StaticNestedClass();
        nested.instanceMethod();
        System.out.println("\nTask B: Inner Class");
        Lab7_NestedClasses outer = new Lab7_NestedClasses();
        InnerClass inner = outer.new InnerClass();
        inner.innerMethod();
        System.out.println("\nTask C: Local Inner Class");
        outer.demonstrateLocalInnerClass();
        System.out.println("\nLab No.: 7");
        System.out.println("Name: Samir Paudel");
        System.out.println("Roll No./Section: 114-079/D");
    }
}
```

Output

```
[sam@arch labs]$  javac Lab7_NestedClasses.java && java Lab7_NestedClasses
--- Nested Classes Demonstration ---

Task A: Static Nested Class
Static Nested Class - Static Method
Static Variable: 10
Static Nested Class - Instance Method

Task B: Non-static Nested Class (Inner Class)
Inner Class - Instance Method
Inner Variable: 20
Accessing outer class variable

Task C: Local Inner Class
Local Inner Class - Method
Local Variable: 30

============================
Lab No.: 7
Name: Samir Paudel
Roll No./Section: 114-079/D
============================
```

Lab 8: Interface Implementation

Title: Create an interface called Shape which has methods area(double x, double y) and perimeter(double x, double y) to compute area and perimeter. Create a class called Rectangle which implements this interface. Declare instance variables as per requirement in class itself. Create instance of the class Rectangle and demonstrate interface implementation.

Theory

Interface: A reference type that defines abstract methods. Uses `interface` keyword. Classes implement interfaces using `implements`. All methods are implicitly `public` and `abstract`.

Key Points:

•Cannot be instantiated

•Supports multiple inheritance variables are `public static final` by default

Code

```java
public class Lab8_InterfaceImplementation {
    interface Shape {
        double area(double x, double y);
        double perimeter(double x, double y);
    }
    static class Rectangle implements Shape {
        double length;
        double width;
        Rectangle(double length, double width) {
            this.length = length;
            this.width = width;
        }
        @Override
        public double area(double x, double y) {
            length = x;
            width = y;
            return length * width;
        }
        @Override
        public double perimeter(double x, double y) {
            length = x;
            width = y;
            return 2 * (length + width);
        }
        void displayDimensions() {
            System.out.println("Rectangle Dimensions:"
                );
            System.out.println("Length: " + length);
            System.out.println("Width: " + width);
        }
    }
    public static void main(String[] args) {
        Rectangle rectangle = new Rectangle(5.0, 4.0);
        rectangle.displayDimensions();
        double area = rectangle.area(5.0, 4.0);
        System.out.println("Area: " + area);
        double perimeter = rectangle.perimeter(5.0,
            4.0);
        System.out.println("Perimeter: " + perimeter);
        Shape shape = new Rectangle(8.0, 6.0);
        System.out.println("\nUsing Shape reference:")
            ;
        System.out.println("Area: " + shape.area(8.0,
            6.0));
        System.out.println("Perimeter: " + shape.
            perimeter(8.0, 6.0));
        System.out.println("\nLab No.: 8");
        System.out.println("Name: Samir Paudel");
        System.out.println("Roll No./Section: 114-079/
            D");
    }
}
```

Output

```
[sam@arch labs]$  javac Lab8_InterfaceImplementation.java && java Lab8_InterfaceImplementation
--- Interface Implementation ---

Rectangle Dimensions:
Length: 5.0
Width: 4.0
Area: 20.0
Perimeter: 18.0

Using Shape interface reference:
Area: 48.0
Perimeter: 28.0

============================
Lab No.: 8
Name: Samir Paudel
Roll No./Section: 114-079/D
============================
```

Lab 9: Exception Handling

Title: WAP to: a) Demonstrate try, catch, and finally blocks in exception handling b) Have multiple catch blocks in exception handling c) Have nested try statements in exception handling d) Demonstrate the use of throw keyword in exception handling e) Demonstrate the use of throws keyword in exception handling f) Illustrate the idea of custom exceptions in Java (user-defined exceptions)

Theory

Exception: An abnormal event disrupting normal program flow. Java provides a robust exception handling framework.

Keywords:

- `try`: Contains risky code
- `catch`: Handles specific exceptions (multiple allowed)
- `finally`: Always executes (cleanup)
- `throw`: Explicitly throws an exception
- `throws`: Declares exceptions a method can throw

Custom Exception: User-defined class extending `Exception` or `RuntimeException`.

Code

```java
import java.io.IOException;
public class Lab9_ExceptionHandling {
    static class CustomException extends Exception {
        CustomException(String message) {
            super(message);
        }
    }
    static void taskTryCatchFinally() {
        System.out.println("\nTask A: Try, Catch, Finally");
        try {
            int x = 10 / 0;
        } catch (ArithmeticException e) {
            System.out.println("Caught: " + e.getMessage());
        } finally {
            System.out.println("Finally executed");
        }
    }
    static void taskMultipleCatch() {
        System.out.println("\nTask B: Multiple Catch");
        try {
            String str = "Hello";
            int num = Integer.parseInt(str);
        } catch (NumberFormatException e) {
            System.out.println("Caught NumberFormat: " +
                e.getMessage());
        } catch (Exception e) {
            System.out.println("Caught Exception: " +
                e.getMessage());
        }
    }
    static void taskNestedTry() {
        System.out.println("\nTask C: Nested Try");
        try {
            try {
                int[] arr = {1, 2, 3};
                System.out.println(arr[5]);
            } catch (ArrayIndexOutOfBoundsException e) {
                System.out.println("Inner catch: " + e
                    );
                throw new Exception("Re-thrown");
            }
        } catch (Exception e) {
            System.out.println("Outer catch: " +
                e.getMessage());
        }
    }
    static void taskThrow() {
        System.out.println("\nTask D: throw keyword");
        try {
            throw new ArithmeticException(
                "Custom arithmetic exception");
        } catch (ArithmeticException e) {
            System.out.println("Caught: " + e.getMessage());
        }
    }
    static void methodWithThrows() throws IOException {
        throw new IOException("I/O Exception");
    }
    static void taskThrows() {
        System.out.println("\nTask E: throws keyword");
        try {
            methodWithThrows();
        } catch (IOException e) {
            System.out.println("Caught: " + e.getMessage());
        }
    }
    static void taskCustomException() {
        System.out.println("\nTask F: Custom Exception");
        try {
            int age = -5;
            if (age < 0) {
                throw new CustomException(
                    "Age cannot be negative");
            }
        } catch (CustomException e) {
            System.out.println("Caught: " + e.getMessage());
```

```
        }
    }
    public static void main(String[] args) {
        taskTryCatchFinally();
        taskMultipleCatch();
        taskNestedTry();
        taskThrow();
        taskThrows();
        taskCustomException();
        System.out.println("\nLab No.: 9");
        System.out.println("Name: Samir Paudel");
        System.out.println("Roll No./Section: 114-079/D");
    }
}
```

Output

```
[sam@arch labs]$  javac Lab9_ExceptionHandling.java && java Lab9_ExceptionHandling
--- Exception Handling Demonstration ---

Task A: Try, Catch, Finally Blocks
Caught ArithmeticException: / by zero
Finally block executed

Task B: Multiple Catch Blocks
Caught NumberFormatException: For input string: "Hello"

Task C: Nested Try Statements
Inner catch: ArrayIndexOutOfBoundsException
Outer catch: Re-throwing exception

Task D: Using throw keyword
Caught: Custom arithmetic exception

Task E: Using throws keyword
Caught: I/O Exception thrown

Task F: Custom Exceptions
Caught CustomException: Age cannot be negative

============================
Lab No.: 9
Name: Samir Paudel
Roll No./Section: 114-079/D
============================
```

Lab 10: Threading

Title: WAP to: a) Demonstrate the process of creating threads by implementing Runnable interface and by extending Thread class b) Show the use of isAlive() and join() methods of Thread c) Demonstrate the process of setting and getting thread priorities d) Illustrate the process of thread synchronization using synchronized method and synchronized block e) Create two threads: first thread prints numbers from 1 to 10 in the interval of 2 seconds and second thread prints numbers from 11 to 20 in the interval of 1 second

Theory

Thread: A lightweight subprocess. Java supports multithreading via `Thread` class and `Runnable` interface.

Key Methods: `start()`, `run()`, `sleep()`, `join()`, `isAlive()`, `setPriority()`, `getPriority()`.

Synchronization: Prevents thread interference on shared resources using `synchronized` keyword (methods or blocks).

Code

```
public class Lab10_Threading {
    static class RunnableThread implements Runnable {
        private String name;
        RunnableThread(String name) {
            this.name = name;
        }
        @Override
        public void run() {
            System.out.println(name + " - Runnable started");
            for (int i = 0; i < 3; i++) {
                System.out.println(name + " - Iteration " + (i + 1));
            }
            System.out.println(name + " - Runnable ended");
        }
    }
    static class ThreadClass extends Thread {
        private String name;
        ThreadClass(String name) {
            this.name = name;
        }
        @Override
        public void run() {
            System.out.println(name + " - Thread started");
            for (int i = 0; i < 3; i++) {
                System.out.println(name + " - Iteration " + (i + 1));
            }
            System.out.println(name + " - Thread ended");
        }
    }
```

```
    static class FirstThread extends Thread {
        @Override
        public void run() {
            System.out.println("\nFirst Thread: 1-10 at 2s");
            for (int i = 1; i <= 10; i++) {
                System.out.println("First Thread: " + i);
                try { Thread.sleep(2000); }
                catch (InterruptedException e) {}
            }
        }
    }
    static class SecondThread extends Thread {
        @Override
        public void run() {
            System.out.println("\nSecond Thread: 11-20 at 1s");
            for (int i = 11; i <= 20; i++) {
                System.out.println("Second Thread: " + i);
                try { Thread.sleep(1000); }
                catch (InterruptedException e) {}
            }
        }
    }
    static void taskIsAliveAndJoin()
        throws InterruptedException {
        System.out.println("\nTask B: isAlive() and join()");
        Thread t = new ThreadClass("JoinThread");
        t.start();
        System.out.println("Is alive? " + t.isAlive());
        t.join();
        System.out.println("After join - alive? " +
            t.isAlive());
    }
    static void taskThreadPriorities() {
        System.out.println("\nTask C: Thread Priorities");
        Thread t1 = new ThreadClass("P1");
        Thread t2 = new ThreadClass("P2");
        t1.setPriority(Thread.MIN_PRIORITY);
        t2.setPriority(Thread.MAX_PRIORITY);
        System.out.println(t1.getName() + " Priority: " +
            t1.getPriority());
        System.out.println(t2.getName() + " Priority: " +
            t2.getPriority());
    }
    static class SharedCounter {
        private int count = 0;
        synchronized void increment() {
            count++;
            System.out.println(
                "Sync method - Count: " + count);
        }
        void incrementBlock() {
            synchronized (this) {
                count++;
                System.out.println(
                    "Sync block - Count: " + count);
            }
        }
    }
    static void taskSynchronization()
        throws InterruptedException {
        System.out.println(
            "\nTask D: Synchronization");
        SharedCounter counter = new SharedCounter();
        Thread t1 = new Thread(() -> {
            for (int i = 0; i < 3; i++)
                counter.increment();
        });
        Thread t2 = new Thread(() -> {
            for (int i = 0; i < 3; i++)
                counter.incrementBlock();
        });
        t1.start(); t2.start();
        t1.join(); t2.join();
    }
    public static void main(String[] args)
        throws InterruptedException {
        System.out.println("Task A: Creating threads");
        Thread t1 = new Thread(
            new RunnableThread("R1"));
        t1.start();
        ThreadClass t2 = new ThreadClass("T1");
        t2.start();
        taskIsAliveAndJoin();
        taskThreadPriorities();
        taskSynchronization();
        System.out.println("\nTask E: Interval threads");
        FirstThread ft = new FirstThread();
        SecondThread st = new SecondThread();
        ft.start();
        st.start();
        ft.join();
        st.join();
        System.out.println("\nLab No.: 10");
        System.out.println("Name: Samir Paudel");
        System.out.println("Roll No./Section: 114-079/D");
    }
}
```

Output

```
[sam@arch labs]$  javac Lab10_Threading.java && timeout 5 java Lab10_Threading
--- Threading Demonstration ---

Task A: Creating threads
--- Using Runnable ---

--- Using Thread class ---

--- Task B: isAlive() and join() methods ---
Is thread alive? true
ThreadClass1 - Thread class started
RunnableThread1 - Runnable thread started
JoinThread - Thread class started
RunnableThread1 - Iteration 1
RunnableThread1 - Iteration 2
RunnableThread1 - Iteration 3
RunnableThread1 - Runnable thread ended
JoinThread - Iteration 1
JoinThread - Iteration 2
JoinThread - Iteration 3
JoinThread - Thread class ended
After join - Is thread alive? false

--- Task C: Thread Priorities ---
ThreadClass1 - Iteration 1
ThreadClass1 - Iteration 2
ThreadClass1 - Iteration 3
ThreadClass1 - Thread class ended
Thread-3 Priority: 1
Thread-4 Priority: 10

--- Task D: Synchronized Method ---

============================
Lab No.: 10
Name: Samir Paudel
Roll No./Section: 114-079/D
============================
```

Lab 11: File I/O and Serialization

Title: WAP to: a) Demonstrate the use of RandomAccessFile class for random accessing of file b) Read text from keyboard and write to a file c) Demonstrate the concept of serialization and deserialization in Java

Theory

RandomAccessFile: Supports random read/write access. Methods like `seek()`, `getFilePointer()`, `readLine()`, `writeBytes()`.

Serialization: Converting object to byte stream using `ObjectOutputStream`. Class must implement `Serializable`.

Deserialization: Reconstructing object from byte stream using `ObjectInputStream`. Requires `serialVersionUID` for versioning.

Code

```java
import java.io.*;
import java.io.RandomAccessFile;
import java.io.Serializable;
public class Lab11_FileIO {
    static void taskRandomAccessFile() {
        System.out.println("\nTask A: RandomAccessFile");
        try {
            RandomAccessFile raf = new RandomAccessFile("randomfile.txt", "rw");
            raf.writeBytes("Hello ");
            raf.writeBytes("World ");
            raf.writeBytes("Java ");
            System.out.println("File pointer at: " + raf.getFilePointer());
            raf.seek(0);
            System.out.println("seek(0): " + raf.readLine());
            raf.seek(6);
            System.out.println("seek(6): " + raf.readLine());
            raf.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
    static void taskKeyboardToFile() {
        System.out.println("\nTask B: Keyboard to file");
        try {
            BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
            FileWriter fw = new FileWriter("keyboard_input.txt");
            BufferedWriter bw = new BufferedWriter(fw);
            System.out.print("Enter text (exit to stop): ");
            String line;
            while (!(line = br.readLine()).equals("
```

```
                exit")) {
                bw.write(line);
                bw.newLine();
            }
            bw.close();
            fw.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
    static class Person implements Serializable {
        private static final long serialVersionUID = 1
            L;
        String name;
        int age;
        Person(String name, int age) {
            this.name = name;
            this.age = age;
        }
        @Override
        public String toString() {
            return "Person{name='" + name +
                "', age=" + age + "}";
        }
    }
    static void taskSerialization() {
        System.out.println("\nTask C: Serialization");
        try {
            Person p = new Person("Samir Paudel", 20);
            FileOutputStream fos = new
                FileOutputStream(
                "person.ser");
            ObjectOutputStream oos = new
                ObjectOutputStream(
                fos);
            oos.writeObject(p);
            oos.close();
            System.out.println("Serialized: " + p);
            FileInputStream fis = new FileInputStream(
                "person.ser");
            ObjectInputStream ois = new
                ObjectInputStream(
                fis);
            Person dp = (Person) ois.readObject();
            ois.close();
            System.out.println("Deserialized: " + dp);
        } catch (IOException | ClassNotFoundException
            e) {
            e.printStackTrace();
        }
    }
    public static void main(String[] args) {
        taskRandomAccessFile();
        taskKeyboardToFile();
        taskSerialization();
        System.out.println("\nLab No.: 11");
        System.out.println("Name: Samir Paudel");
        System.out.println("Roll No./Section: 114-079/
            D");
    }
}
```

Output

```
[sam@arch labs]$  javac Lab11_FileIO.java && java Lab11_FileIO
--- File I/O Operations ---

Task A: RandomAccessFile
File pointer at: 17
After seek(0), reading: Hello World Java
After seek(6), reading: World Java
RandomAccessFile operations completed

Task C: Serialization and Deserialization
Object serialized: Person{name='Samir Paudel', age=20}
Object deserialized: Person{name='Samir Paudel', age=20}

============================
Lab No.: 11
Name: Samir Paudel
Roll No./Section: 114-079/D
============================
```

Lab 12: Swing - Colored Buttons

Title: Using swing components, design a form with three buttons with captions ”RED,” ”BLUE,” and ”GREEN,” respectively. Then write a program to handle the event such that when the user clicks the button, the color of that button will be the same as its caption.

Theory

Swing: GUI toolkit with platform-independent components. `JFrame` is the main window, `JButton` is a clickable component.

Event Handling: Register `ActionListener` via `addActionListener()`. The `actionPerformed()` method handles clicks.

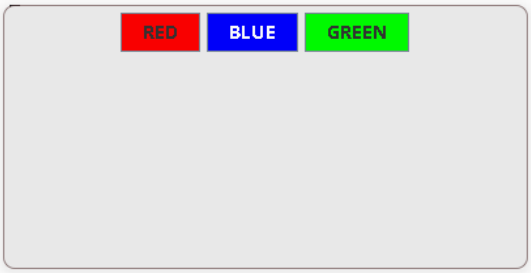
Button Appearance: Use `setBackground()`, `setOpaque(true)`, and `setBorderPainted(true)` for colored buttons.

Code

```java
import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
public class Lab12_SwingColorButtons extends JFrame {
    private JButton redButton;
    private JButton blueButton;
    private JButton greenButton;
    Lab12_SwingColorButtons() {
        setTitle("Colored Buttons");
        setSize(400, 200);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        JPanel panel = new JPanel();
        panel.setLayout(new FlowLayout());
        redButton = new JButton("RED");
        redButton.setFont(new Font("Arial", Font.BOLD, 14));
        redButton.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                redButton.setBackground(Color.RED);
                redButton.setOpaque(true);
                redButton.setBorderPainted(true);
            }
        });
        panel.add(redButton);
        blueButton = new JButton("BLUE");
        blueButton.setFont(new Font("Arial", Font.BOLD, 14));
        blueButton.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                blueButton.setBackground(Color.BLUE);
                blueButton.setOpaque(true);
                blueButton.setBorderPainted(true);
                blueButton.setForeground(Color.WHITE);
            }
        });
        panel.add(blueButton);
        greenButton = new JButton("GREEN");
        greenButton.setFont(new Font("Arial", Font.BOLD, 14));
        greenButton.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                greenButton.setBackground(Color.GREEN);
                greenButton.setOpaque(true);
                greenButton.setBorderPainted(true);
            }
        });
        panel.add(greenButton);
        add(panel);
        setVisible(true);
    }
    public static void main(String[] args) {
        SwingUtilities.invokeLater(new Runnable() {
            @Override
            public void run() {
                new Lab12_SwingColorButtons();
            }
        });
        System.out.println("Lab No.: 12");
        System.out.println("Name: Samir Paudel");
        System.out.println("Roll No./Section: 114-079/D");
    }
}
```

Output

```
[sam@arch java]$  cd /home/sam/Github/semester-works/semester-7
lorButtons
============================
Lab No.: 12
Name: Samir Paudel
Roll No./Section: 114-079/D
============================
```



Lab 13: Simple GUI - Text Field

Title: Write a simple GUI program that displays ”Hello World” in a text field. The program should display output in option dialog if user clicks a button.

Theory

JTextField: Single-line text component. Can be set non-editable with `setEditable(false)`.

JOptionPane: Standard dialog box for messages. `showMessageDialog()` displays a simple message with an OK button.

Layout: `JPanel` with default FlowLayout arranges components in a row.

Code

```java
import javax.swing.*;
public class Lab13_SimpleGUI extends JFrame {
    private JTextField textField;
    private JButton button;
    Lab13_SimpleGUI() {
        setTitle("Simple GUI");
        setSize(300, 150);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)
            ;
        setLocationRelativeTo(null);
        JPanel panel = new JPanel();
        textField = new JTextField("Hello World", 20);
        textField.setEditable(false);
        panel.add(textField);
        button = new JButton("Click Me");
        button.addActionListener(e -> {
            JOptionPane.showMessageDialog(
                Lab13_SimpleGUI.this,
                "Output: " + textField.getText(),
                "Message",
                JOptionPane.INFORMATION_MESSAGE);
        });
        panel.add(button);
        add(panel);
        setVisible(true);
    }
    public static void main(String[] args) {
        SwingUtilities.invokeLater(new Runnable() {
            @Override
            public void run() {
                new Lab13_SimpleGUI();
            }
        });
        System.out.println("Lab No.: 13");
        System.out.println("Name: Samir Paudel");
        System.out.println("Roll No./Section: 114-079/
            D");
    }
}
```

Output

```
[sam@arch labs]$ javac Lab13_SimpleGUI.java && java Lab13_SimpleGUI
==============================
Lab No.: 13
Name: Samir Paudel
Roll No./Section: 114-079/D
==============================
```

Lab 14: Text Area with File Operations

Title: Write a simple GUI program that has text area and two buttons ”Read” and ”Write”. Whenever write button is clicked, the text in text area should be written to the file named ”userinput.txt” and when read button is clicked, the text in file should be shown in text area.

Theory

JTextArea: Multi-line text component. Wrap text with `setLineWrap(true)`. Add scrolling via `JScrollPane`.

File I/O in GUI: `FileWriter/BufferedWriter` write text, `FileReader/BufferedReader` read text. Use `JOptionPane` for feedback messages.

Code

```java
import javax.swing.*;
import java.awt.event.*;
import java.io.*;
public class Lab14_TextAreaFileOps extends JFrame {
    private JTextArea textArea;
    private JButton readButton;
    private JButton writeButton;
    private static final String FILE_NAME =
        "userinput.txt";
    Lab14_TextAreaFileOps() {
        setTitle("Text Area with File Operations");
        setSize(400, 300);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)
            ;
        setLocationRelativeTo(null);
        JPanel panel = new JPanel();
        panel.setLayout(
            new BoxLayout(panel, BoxLayout.Y_AXIS));
        textArea = new JTextArea(10, 40);
        textArea.setLineWrap(true);
        textArea.setWrapStyleWord(true);
        JScrollPane scrollPane = new JScrollPane(
            textArea);
        panel.add(scrollPane);
        JPanel buttonPanel = new JPanel();
        readButton = new JButton("Read");
        readButton.addActionListener(e -> readFromFile
            ());
        buttonPanel.add(readButton);
        writeButton = new JButton("Write");
        writeButton.addActionListener(e -> writeToFile
            ());
        buttonPanel.add(writeButton);
        panel.add(buttonPanel);
        add(panel);
        setVisible(true);
    }
    private void writeToFile() {
        try {
            FileWriter fw = new FileWriter(FILE_NAME);
            BufferedWriter bw = new BufferedWriter(fw)
                ;
            bw.write(textArea.getText());
            bw.close();
            JOptionPane.showMessageDialog(this,
                "Written to " + FILE_NAME);
        } catch (IOException ex) {
            JOptionPane.showMessageDialog(this,
                "Error: " + ex.getMessage());
        }
    }
    private void readFromFile() {
        try {
            FileReader fr = new FileReader(FILE_NAME);
            BufferedReader br = new BufferedReader(fr)
                ;
            StringBuilder content = new StringBuilder
                ();
            String line;
            while ((line = br.readLine()) != null) {
                content.append(line).append("\n");
            }
            br.close();
            textArea.setText(content.toString());
            JOptionPane.showMessageDialog(this,
                "Read from " + FILE_NAME);
        } catch (IOException ex) {
            JOptionPane.showMessageDialog(this,
                "Error: " + ex.getMessage());
        }
    }
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() ->
            new Lab14_TextAreaFileOps());
        System.out.println("Lab No.: 14");
        System.out.println("Name: Samir Paudel");
        System.out.println("Roll No./Section: 114-079/
            D");
    }
}
```

Output

Lab 15: Menu Bar

Title: Write a program to create a menu named ”File” with menu items ”New,” ”Save,” and ”Exit”.

Theory

JMenuBar: Container for menus. Attached to JFrame via `setJMenuBar()`.

JMenu: Dropdown menu. **JMenuItem:** Individual clickable items with `ActionListener`.

Hierarchy: JFrame → JMenuBar → JMenu → JMenuItem.

Code

```java
import javax.swing.*;
public class Lab15_MenuBar extends JFrame {
    Lab15_MenuBar() {
        setTitle("Menu Bar");
        setSize(400, 300);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)
            ;
        setLocationRelativeTo(null);
        JMenuBar menuBar = new JMenuBar();
        JMenu fileMenu = new JMenu("File");
        JMenuItem newItem = new JMenuItem("New");
        newItem.addActionListener(e ->
            JOptionPane.showMessageDialog(this,
                "New file selected"));
        fileMenu.add(newItem);
        JMenuItem saveItem = new JMenuItem("Save");
        saveItem.addActionListener(e ->
            JOptionPane.showMessageDialog(this,
                "File saved"));
        fileMenu.add(saveItem);
        fileMenu.addSeparator();
        JMenuItem exitItem = new JMenuItem("Exit");
        exitItem.addActionListener(e -> System.exit(0)
            );
        fileMenu.add(exitItem);
        menuBar.add(fileMenu);
        setJMenuBar(menuBar);
        JLabel label = new JLabel(
            "Menu Bar Example");
        add(label);
        setVisible(true);
    }
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() ->
            new Lab15_MenuBar());
        System.out.println("Lab No.: 15");
        System.out.println("Name: Samir Paudel");
        System.out.println("Roll No./Section: 114-079/
            D");
    }
}
```

Output

Lab 16: JTextField with KeyListener

Title: Write a Java Swing program with a JTextField. Implement a KeyListener such that: a) Only numeric digits (0-9) can be typed into the JTextField b) If any other key is typed (e.g., a letter or symbol), it should be ignored (i.e., not appear in the text field)

Theory

KeyListener: Captures keyboard events. Methods: `keyPressed()`, `keyReleased()`, `keyTyped()`.

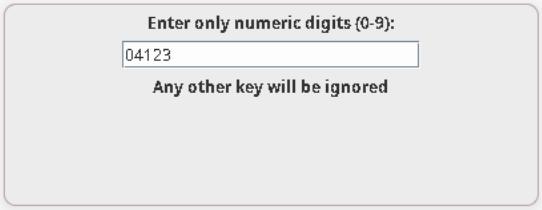
Event Consumption: `e.consume()` prevents the character from appearing in the text field. Useful for input validation.

Code

```java
import javax.swing.*;
import java.awt.event.KeyEvent;
import java.awt.event.KeyListener;
public class Lab16_JTextFieldKeyListener
    extends JFrame {
    private JTextField textField;
    Lab16_JTextFieldKeyListener() {
        setTitle("Numeric Digits Only");
        setSize(400, 150);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)
            ;
        setLocationRelativeTo(null);
        JPanel panel = new JPanel();
        JLabel label = new JLabel(
            "Enter only numeric digits (0-9):");
        panel.add(label);
        textField = new JTextField(20);
        textField.addKeyListener(new KeyListener() {
            @Override
            public void keyPressed(KeyEvent e) {}
            @Override
            public void keyReleased(KeyEvent e) {}
            @Override
            public void keyTyped(KeyEvent e) {
                char c = e.getKeyChar();
                if (!Character.isDigit(c)) {
                    e.consume();
                }
            }
        });
        panel.add(textField);
        JLabel infoLabel = new JLabel(
            "Non-numeric keys ignored");
        panel.add(infoLabel);
        add(panel);
        setVisible(true);
    }
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() ->
            new Lab16_JTextFieldKeyListener());
        System.out.println("Lab No.: 16");
        System.out.println("Name: Samir Paudel");
        System.out.println("Roll No./Section: 114-079/
            D");
    }
}
```

Output

```
============================
Lab No.: 16
Name: Samir Paudel
Roll No./Section: 114-079/D
============================
```



Lab 17: JCheckBox and JRadioButtons

Title: Create a GUI with one JCheckBox labeled ”Enable Extra Options” and three JRadioButtons (e.g., ”Option A,” ”Option B,” ”Option C”) grouped in a ButtonGroup. a) Initially, the radio buttons should be disabled b) Implement an ItemListener for the JCheckBox. When the checkbox is selected, the radio buttons should become enabled. When it’s deselected, they should become disabled c) Implement an ItemListener for the radio buttons. When a radio button is selected, print its label (e.g., ”Option A selected”) to the console

Theory

JCheckBox: Toggleable option that fires `ItemEvent` on state change.

JRadioButton: Mutually exclusive within a `ButtonGroup`. Only one can be selected.

ItemListener: Responds to selection/deselection via `itemStateChanged()`. Check `ItemEvent.SELECTED` vs `ItemEvent.DESELECTED`.

Code

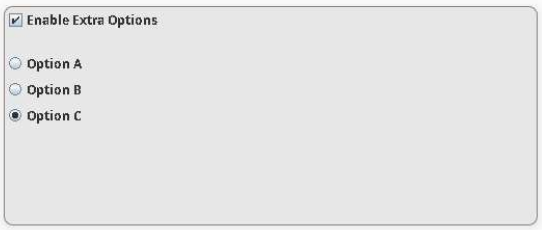
```java
import javax.swing.*;
import java.awt.event.ItemEvent;
import java.awt.event.ItemListener;
public class Lab17_CheckBoxRadioButtons
    extends JFrame {
    private JCheckBox checkBox;
    private JRadioButton optionA;
    private JRadioButton optionB;
    private JRadioButton optionC;
    private ButtonGroup buttonGroup;
    Lab17_CheckBoxRadioButtons() {
        setTitle("CheckBox and RadioButtons");
        setSize(400, 250);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)
            ;
        setLocationRelativeTo(null);
        JPanel panel = new JPanel();
        panel.setLayout(
            new BoxLayout(panel, BoxLayout.Y_AXIS));
        checkBox = new JCheckBox(
            "Enable Extra Options");
        checkBox.addItemListener(new ItemListener() {
            @Override
            public void itemStateChanged(ItemEvent e)
                {
                boolean selected = e.getStateChange()
                    == ItemEvent.SELECTED;
                optionA.setEnabled(selected);
                optionB.setEnabled(selected);
                optionC.setEnabled(selected);
                if (!selected)
                    buttonGroup.clearSelection();
                System.out.println("Radio buttons " +
                    (selected ? "enabled" : "disabled"
                        ));
            }
        });
        panel.add(checkBox);
        optionA = new JRadioButton("Option A");
        optionA.setEnabled(false);
        optionA.addItemListener(e -> {
            if (e.getStateChange() == ItemEvent.
                SELECTED)
                System.out.println("Option A selected"
                    );
        });
        panel.add(optionA);
        optionB = new JRadioButton("Option B");
        optionB.setEnabled(false);
        optionB.addItemListener(e -> {
            if (e.getStateChange() == ItemEvent.
                SELECTED)
                System.out.println("Option B selected"
                    );
        });
        panel.add(optionB);
        optionC = new JRadioButton("Option C");
        optionC.setEnabled(false);
        optionC.addItemListener(e -> {
            if (e.getStateChange() == ItemEvent.
                SELECTED)
                System.out.println("Option C selected"
                    );
        });
        panel.add(optionC);
        buttonGroup = new ButtonGroup();
        buttonGroup.add(optionA);
        buttonGroup.add(optionB);
        buttonGroup.add(optionC);
        add(panel);
        setVisible(true);
    }
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() ->
            new Lab17_CheckBoxRadioButtons());
        System.out.println("Lab No.: 17");
        System.out.println("Name: Samir Paudel");
        System.out.println("Roll No./Section: 114-079/
            D");
    }
}
```

Output

Lab 18: JDBC - MOVIE Table Operations

Title: Assume a table MOVIE(id, title, genre). Using JDBC, perform the following queries: a) Add any three records to the MOVIE table b) Using a prepared statement, update the genre to ”Comedy” where title = ”Jatra”

```
[sam@arch java]$ cd /home/sam/Github/semester-works/semester-7/java/labs && javac Lab17_CheckBoxRadioButtons.java && jav
kBoxRadioButtons
============================
Lab No.: 17
Name: Samir Paudel
Roll No./Section: 114-079/D
============================
Radio buttons enabled
Option A selected
Option B selected
Option C selected
```



Theory

JDBC: Java Database Connectivity API. Steps: Load driver, connect, create statement, execute query, process results, close.

PreparedStatement: Parameterized query using ? placeholders. Prevents SQL injection.

CRUD Operations: INSERT (create), SELECT (read), UPDATE (modify), DELETE (remove).

Code

```java
import java.sql.*;
public class Lab18_JDBC_MovieTable {
    static final String DB_URL = "jdbc:mariadb://localhost:3306/java_lab";
    static final String USER = "root";
    static final String PASS = "password";
    static final String DRIVER = "org.mariadb.jdbc.Driver";
    public static void main(String[] args) {
        Connection conn = null;
        try {
            Class.forName(DRIVER);
            System.out.println("JDBC Driver loaded");
            conn = DriverManager.getConnection(DB_URL, USER, PASS);
            System.out.println("Connected to database");
            createTables(conn);
            insertDirectors(conn);
            insertMovies(conn);
            System.out.println("\n=== Movies with Directors (SQL JOIN) ===");
            displayMoviesWithDirectors(conn);
            System.out.println("\n=== Updating Jatra genre to Comedy ===");
            updateMovieGenre(conn, "Jatra", "Comedy");
            System.out.println("\n=== Updated Movie List ===");
            displayMoviesWithDirectors(conn);
            System.out.println("\nLab No.: 18\nName: Samir Paudel\nRoll No./Section: 114-079/D");
        } catch (ClassNotFoundException e) {
            System.out.println("JDBC Driver Error: " + e.getMessage());
        } catch (SQLException e) {
            System.out.println("Database Error: " + e.getMessage());
        } finally {
            try { if (conn != null) conn.close(); } catch (SQLException e) {}
        }
    }
    static void createTables(Connection conn) throws SQLException {
        Statement stmt = conn.createStatement();
        stmt.execute("DROP TABLE IF EXISTS MOVIE");
        stmt.execute("DROP TABLE IF EXISTS DIRECTOR");
        stmt.execute("CREATE TABLE DIRECTOR (" +
            "id INT PRIMARY KEY AUTO_INCREMENT, " +
            "name VARCHAR(100) NOT NULL)");
        stmt.execute("CREATE TABLE MOVIE (" +
            "id INT PRIMARY KEY AUTO_INCREMENT, " +
            "title VARCHAR(100) NOT NULL, " +
            "genre VARCHAR(50) NOT NULL, " +
            "rating DOUBLE NOT NULL, " +
            "director_id INT NOT NULL, " +
            "FOREIGN KEY (director_id) REFERENCES DIRECTOR(id))");
        stmt.close();
    }
    static void insertDirectors(Connection conn) throws SQLException {
        String sql = "INSERT INTO DIRECTOR (name) VALUES (?)";
        PreparedStatement pstmt = conn.prepareStatement(sql);
        String[] directors = {"Anurag Kashyap", "Vikas Bahl", "James Cameron"};
        for (String d : directors) { pstmt.setString(1, d); pstmt.executeUpdate(); }
        pstmt.close();
    }
    static void insertMovies(Connection conn) throws SQLException {
        String sql = "INSERT INTO MOVIE (title, genre, rating, director_id) VALUES (?, ?, ?, ?)";
        PreparedStatement pstmt = conn.prepareStatement(sql);
        Object[][] movies = {
            {"Inception", "Science Fiction", 8.8, 3},
            {"Jatra", "Drama", 7.5, 1},
            {"Avatar", "Science Fiction", 7.8, 3}
        };
        for (Object[] m : movies) {
            pstmt.setString(1, (String) m[0]);
            pstmt.setString(2, (String) m[1]);
            pstmt.setDouble(3, (Double) m[2]);
            pstmt.setInt(4, (Integer) m[3]);
            pstmt.executeUpdate();
        }
        pstmt.close();
    }
    static void displayMoviesWithDirectors(Connection conn) throws SQLException {
        String query = "SELECT m.id, m.title, m.genre, m.rating, d.name AS director_name " +
            "FROM MOVIE m JOIN DIRECTOR d ON m.director_id = d.id ORDER BY m.id";
        Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery(query);
        System.out.println("ID | Title | Genre | Rating | Director");
```

```
        System.out.println("
            ---+------------+--------------------+--------+-
            ");
        while (rs.next()) {
            System.out.printf("%d  | %-10s | %-18s |
                %.1f   | %s%n",
                rs.getInt("id"), rs.getString("title")
                    ,
                rs.getString("genre"), rs.getDouble("
                    rating"),
                rs.getString("director_name"));
        }
        rs.close(); stmt.close();
    }
    static void updateMovieGenre(Connection conn,
        String title, String newGenre) throws
        SQLException {
        String sql = "UPDATE MOVIE SET genre = ? WHERE
            title = ?";
        PreparedStatement pstmt = conn.
            prepareStatement(sql);
        pstmt.setString(1, newGenre);
        pstmt.setString(2, title);
        int rows = pstmt.executeUpdate();
        System.out.println(rows + " record(s) updated"
            );
        pstmt.close();
    }
}
```

Output

```
[sam@arch labs]$ javac -cp mariadb-java-client.jar Lab18_JDBC_MovieTable.java && java -cp .:mariadb-java-client.jar Lab18_JDBC_MovieTable
✓ JDBC Driver loaded
✓ Connected to database: jdbc:mariadb://localhost:3306/java_lab
✓ Old tables dropped
✓ DIRECTOR table created
✓ MOVIE table created with foreign key
✓ 3 Directors inserted
✓ 3 Movies inserted

=== Movies with Directors (SQL JOIN) ===

ID | Title      | Genre              | Rating | Director
---+------------+--------------------+--------+-----------
1  | Inception  | Science Fiction    | 8.8    | James Cameron
2  | Jatra      | Drama              | 7.5    | Anurag Kashyap
3  | Avatar     | Science Fiction    | 7.8    | James Cameron

=== Updating Jatra genre to Comedy ===
✓ 1 record(s) updated

=== Updated Movie List ===

ID | Title      | Genre              | Rating | Director
---+------------+--------------------+--------+-----------
1  | Inception  | Science Fiction    | 8.8    | James Cameron
2  | Jatra      | Comedy             | 7.5    | Anurag Kashyap
3  | Avatar     | Science Fiction    | 7.8    | James Cameron

--- Lab Information ---
Lab No.: 18
Name: Samir Paudel
Roll No./Section: 114-079/D
✓ Connection closed
[sam@arch labs]$
```

Lab 19: Scrollable ResultSet Navigation

Title: Write a program to create a scrollable ResultSet and navigate it to the last row, then to the first row, and then to the third row. Consider MOVIE table.

Theory

ResultSet Types: `TYPE_FORWARD_ONLY` (default), `TYPE_SCROLL_INSENSITIVE` (scrollable, no live updates), `TYPE_SCROLL_SENSITIVE` (scrollable, reflects changes).

Navigation Methods: `first()`, `last()`, `absolute(n)`, `relative(n)`, `beforeFirst()`, `next()`, `previous()`.

Code

```java
import java.sql.*;
public class Lab19_ScrollableResultSet {
    static final String DB_URL = "jdbc:mariadb://localhost:3306/movie_db";
    static final String USER = "root";
    static final String PASS = "password";
    public static void main(String[] args) {
        System.out.println("--- Scrollable ResultSet Navigation ---");
        String sql = "SELECT * FROM MOVIE";
        try (Connection conn = DriverManager.getConnection(DB_URL, USER, PASS);
             Statement stmt = conn.createStatement(ResultSet.TYPE_SCROLL_INSENSITIVE, ResultSet.CONCUR_READ_ONLY);
             ResultSet rs = stmt.executeQuery(sql)) {
            System.out.println("\nNavigating to Last Row:");
            if (rs.last()) System.out.println("Last row - ID: " + rs.getInt("id") + ", Title: " + rs.getString("title") + ", Genre: " + rs.getString("genre"));
            System.out.println("\nNavigating to First Row:");
            if (rs.first()) System.out.println("First row - ID: " + rs.getInt("id") + ", Title: " + rs.getString("title") + ", Genre: " + rs.getString("genre"));
            System.out.println("\nNavigating to Third Row:");
            if (rs.absolute(3)) System.out.println("Third row - ID: " + rs.getInt("id") + ", Title: " + rs.getString("title") + ", Genre: " + rs.getString("genre"));
            else System.out.println("Third row does not exist");
            System.out.println("\nAll records from ResultSet:");
            rs.beforeFirst();
            while (rs.next()) System.out.println("ID: " + rs.getInt("id") + ", Title: " + rs.getString("title") + ", Genre: " + rs.getString("genre"));
        } catch (SQLException e) { e.printStackTrace(); }
        System.out.println("\nLab No.: 19\nName: Samir Paudel\nRoll No./Section: 114-079/D");
    }
}
```

Output

```
[sam@arch labs]$ javac -cp mariadb-java-client.jar Lab19_ScrollableResultSet.java && java -cp .:mariadb-java-client.jar Lab19_ScrollableResultSet
--- Scrollable ResultSet Navigation ---

Navigating to Last Row:
Last row - ID: 5, Title: Titanic, Genre: Romance

Navigating to First Row:
First row - ID: 1, Title: Inception, Genre: Science Fiction

Navigating to Third Row:
Third row - ID: 3, Title: Avatar, Genre: Science Fiction

All records from ResultSet:
ID: 1, Title: Inception, Genre: Science Fiction
ID: 2, Title: Jatra, Genre: Drama
ID: 3, Title: Avatar, Genre: Science Fiction
ID: 4, Title: Interstellar, Genre: fiction
ID: 5, Title: Titanic, Genre: Romance

============================
Lab No.: 19
Name: Samir Paudel
Roll No./Section: 114-079/D
============================
[sam@arch labs]$
```

Lab 20: Updatable ResultSet

Title: Write a program that retrieves data from a movie table using an updatable ResultSet. The program should find a movie with title = 'interstellar' and update its genre to fiction directly through the ResultSet object.

Theory

Updatable ResultSet: Created with `CONCUR_UPDATABLE` concurrency mode. Allows in-place updates through ResultSet.

Update Methods: `updateString(col, val)`, `updateInt(col, val)`, then `updateRow()` to persist.

Concurrency Modes: `CONCUR_READ_ONLY` (default), `CONCUR_UPDATABLE` (allows updates).

Code

```java
import java.sql.*;
public class Lab20_UpdatableResultSet {
    static final String DB_URL = "jdbc:mariadb://localhost:3306/movie_db";
    static final String USER = "root";
    static final String PASS = "password";
    public static void main(String[] args) {
        System.out.println("--- Updatable ResultSet ---");
        String sql = "SELECT * FROM MOVIE";
        try (Connection conn = DriverManager.getConnection(DB_URL, USER, PASS);
             Statement stmt = conn.createStatement(
                 ResultSet.TYPE_SCROLL_SENSITIVE,
                 ResultSet.CONCUR_UPDATABLE);
             ResultSet rs = stmt.executeQuery(sql)) {
            System.out.println("\nSearching for movie with title 'interstellar':");
            boolean found = false;
            while (rs.next()) {
                String title = rs.getString("title");
                System.out.println("Current row - ID: " + rs.getInt("id") +
                    ", Title: " + title + ", Genre: " + rs.getString("genre"));
                if (title.equalsIgnoreCase("interstellar")) {
                    found = true;
                    System.out.println("\nFound 'interstellar' movie!");
                    rs.updateString("genre", "fiction");
                    rs.updateRow();
                    System.out.println("Updated genre to 'fiction'");
                    System.out.println("Updated row - ID: " + rs.getInt("id") +
                        ", Title: " + rs.getString("title") + ", Genre: " + rs.getString("genre"));
                    break;
                }
            }
            if (!found) {
                System.out.println("Movie 'interstellar' not found");
                rs.beforeFirst();
                while (rs.next()) System.out.println("ID: " + rs.getInt("id") +
                    ", Title: " + rs.getString("title") + ", Genre: " + rs.getString("genre"));
            }
        } catch (SQLException e) { e.printStackTrace(); }
        System.out.println("\nLab No.: 20\nName: Samir Paudel\nRoll No./Section: 114-079/D");
    }
}
```

Output

```
[sam@arch labs]$ javac -cp mariadb-java-client.jar Lab20_UpdatableResultSet.java && java -cp .:mariadb-java-client.jar Lab20_UpdatableResultSet
--- Updatable ResultSet ---

Searching for movie with title 'interstellar':
Current row - ID: 1, Title: Inception, Genre: Science Fiction
Current row - ID: 2, Title: Jatra, Genre: Drama
Current row - ID: 3, Title: Avatar, Genre: Science Fiction
Current row - ID: 4, Title: Interstellar, Genre: fiction

Found 'interstellar' movie!
Updated genre to 'fiction'
Updated row - ID: 4, Title: Interstellar, Genre: fiction

============================
Lab No.: 20
Name: Samir Paudel
Roll No./Section: 114-079/D
============================
[sam@arch labs]$
```

Lab 21: TCP Sockets - One-Way Chat

Title: Write a simple one-way chat application using TCP sockets. Server Code: Listens for a connection, reads a line of text from the client, and prints it. Client Code: Connects to the server, reads a line of text from the user's console, and sends it to the server.

Theory

TCP: Connection-oriented protocol providing reliable communication. Uses `ServerSocket` (server) and `Socket` (client).

Process: Server creates `ServerSocket` and calls `accept()` (blocking). Client creates `Socket` to server. Data flows via streams (`InputStreamReader`, `PrintWriter`).

Code

```java
import java.io.*;
import java.net.*;
import java.util.Scanner;
public class Lab21_TCPDemo {
    static class SimpleServer extends Thread {
        public void run() {
            try {
                ServerSocket serverSocket =
                    new ServerSocket(5000);
                System.out.println(
                    "Server on port 5000");
                Socket socket =
                    serverSocket.accept();
                System.out.println("Client: " +
                    socket.getInetAddress());
                BufferedReader reader =
                    new BufferedReader(
                        new InputStreamReader(
                            socket.getInputStream()));
                String msg = reader.readLine();
                System.out.println(
                    "Received: " + msg);
                reader.close();
                socket.close();
                serverSocket.close();
            } catch (IOException e) {
                e.printStackTrace();
            }
        }
    }
    static class SimpleClient {
        public static void main(String[] args) {
            try {
                Thread.sleep(1000);
                Socket socket = new Socket(
                    "localhost", 5000);
                System.out.println("Connected");
                Scanner sc = new Scanner(System.in);
                System.out.print("Enter message: ");
                String msg = sc.nextLine();
                PrintWriter writer = new PrintWriter(
                    socket.getOutputStream(), true);
                writer.println(msg);
                System.out.println("Sent: " + msg);
                writer.close();
                socket.close();
            } catch (Exception e) {
                e.printStackTrace();
            }
        }
    }
    public static void main(String[] args)
        throws InterruptedException {
        SimpleServer server = new SimpleServer();
        server.start();
        Thread.sleep(500);
        SimpleClient.main(new String[]{});
        server.join();
        System.out.println("\nLab No.: 21");
        System.out.println("Name: Samir Paudel");
        System.out.println("Roll No./Section: 114-079/
            D");
    }
}
```

Output

```
[sam@arch labs]$  echo "=== Lab 21: TCP Demo ===" && javac Lab21_TCPDemo.java && java Lab21_TCPDemo
=== Lab 21: TCP Demo ===
--- TCP Socket Server ---
Server waiting for connection on port 5000
--- TCP Socket Client ---
Connected to server
Message sent: Hello from Client!
Client connected from: /127.0.0.1
Message received from client: Hello from Client!

============================
Lab No.: 21
Name: Samir Paudel
Roll No./Section: 114-079/D
============================
```

Lab 22: UDP Echo Server and Client

Title: Create a simple UDP Echo Server and Client. Server Code: Listens on a port, receives a packet, and sends the exact same data back to the sender's address and port. Client Code: Sends a message to the server and waits to receive the echo, then prints it.

Theory

UDP: Connectionless protocol. Uses `DatagramSocket` for sending/receiving and `DatagramPacket` for data with address info.

Echo Pattern: Client sends packet to server. Server receives, creates response packet with same data and client's address/port, sends back.

Code

```java
import java.net.*;
public class Lab22_UDPDemo {
    static class EchoServer extends Thread {
        public void run() {
            try {
                DatagramSocket socket =
                    new DatagramSocket(5001);
                System.out.println(
                    "UDP Server on port 5001");
                byte[] buf = new byte[1024];
                DatagramPacket recv =
                    new DatagramPacket(
                        buf, buf.length);
                socket.receive(recv);
                String msg = new String(
                    recv.getData(), 0,
                    recv.getLength());
                System.out.println(
                    "Received: " + msg);
                byte[] sendData = msg.getBytes();
                DatagramPacket send =
                    new DatagramPacket(
                        sendData, sendData.length,
                        recv.getAddress(),
                        recv.getPort());
                socket.send(send);
                System.out.println("Echo sent");
                socket.close();
            } catch (Exception e) {
                e.printStackTrace();
            }
        }
    }
    static class EchoClient {
        public static void main(String[] args) {
            try {
                Thread.sleep(1000);
                DatagramSocket socket =
                    new DatagramSocket();
                String msg = "Hello UDP Server!";
                byte[] data = msg.getBytes();
                InetAddress addr =
                    InetAddress.getByName(
                        "localhost");
                DatagramPacket send =
                    new DatagramPacket(
                        data, data.length,
                        addr, 5001);
                socket.send(send);
                System.out.println("Sent: " + msg);
                byte[] buf = new byte[1024];
                DatagramPacket recv =
                    new DatagramPacket(
                        buf, buf.length);
                socket.receive(recv);
                String echo = new String(
                    recv.getData(), 0,
                    recv.getLength());
                System.out.println(
                    "Echo: " + echo);
                socket.close();
            } catch (Exception e) {
                e.printStackTrace();
            }
        }
    }
    public static void main(String[] args)
        throws InterruptedException {
        EchoServer server = new EchoServer();
        server.start();
        Thread.sleep(500);
        EchoClient.main(new String[]{});
        server.join();
        System.out.println("\nLab No.: 22");
        System.out.println("Name: Samir Paudel");
        System.out.println("Roll No./Section: 114-079/
            D");
    }
}
```

Output

```
=== Lab 22: UDP Demo ===
--- UDP Echo Server ---
Server listening on port 5001
--- UDP Echo Client ---
Message sent: Hello UDP Server!
Message received: Hello UDP Server!
Echo sent back to client
Echo received: Hello UDP Server!

============================
Lab No.: 22
Name: Samir Paudel
Roll No./Section: 114-079/D
============================
```

Lab 23: URL Object Handling

Title: Write a Java program that creates a URL object for https://www.example.com/index.html?user=test and prints its protocol, host, and file path.

Theory

URL: Uniform Resource Locator identifies a web resource. The `java.net.URL` class parses URLs and provides access to components: protocol, host, port, path, query, file.

Methods: `getProtocol()`, `getHost()`, `getPort()`, `getPath()`, `getQuery()`, `getFile()`.

Code

```java
import java.net.*;
public class Lab23_URLObject {
    public static void main(String[] args) {
        String urlString =
            "https://www.example.com/" +
            "index.html?user=test";
        try {
            URL url = new URL(urlString);
            System.out.println("URL: " + urlString);
            System.out.println(
                "Protocol: " + url.getProtocol());
            System.out.println(
                "Host: " + url.getHost());
            System.out.println(
                "File: " + url.getFile());
            System.out.println(
                "Port: " + url.getPort());
            System.out.println(
                "Path: " + url.getPath());
            System.out.println(
                "Query: " + url.getQuery());
        } catch (MalformedURLException e) {
            System.out.println(
                "Invalid URL: " + e.getMessage());
        }
        System.out.println("\nLab No.: 23");
        System.out.println("Name: Samir Paudel");
        System.out.println("Roll No./Section: 114-079/
            D");
    }
}
```

Output

```
[sam@arch labs]$  javac Lab3_StaticAndFinal.java Lab23_URLObject.java && java Lab23_URLObject
Note: Lab23_URLObject.java uses or overrides a deprecated API.
Note: Recompile with -Xlint:deprecation for details.
--- URL Object Handling ---

URL: https://www.example.com/index.html?user=test
-----------------------------------
Protocol: https
Host: www.example.com
File path: /index.html?user=test

Additional Information:
Port: -1
Default Port: 443
Path: /index.html
Query: user=test
Authority: www.example.com

============================
Lab No.: 23
Name: Samir Paudel
Roll No./Section: 114-079/D
============================
```

Lab 24: Email Using JavaMail API

Title: Write a Java program to send a simple email using the JavaMail API. Assume you have a Session object properly configured. The email should be from sender@test.com to receiver@test.com with the subject ”Test Mail”.

Theory

JavaMail API: Library for email operations. Uses `Session`, `MimeMessage`, and `Transport`.

SMTP Configuration: Host (smtp.gmail.com), Port (587 for TLS), authentication required, STARTTLS enabled.

Requirements: mail.jar, activation.jar in classpath. For Gmail, use App Password with 2FA.

Code

```java
import javax.mail.*;
import javax.mail.internet.*;
import java.util.Properties;
public class Lab24_SendEmail {
    public static void main(String[] args) {
        Properties props = new Properties();
        props.put("mail.smtp.host", "smtp.gmail.com");
        props.put("mail.smtp.port", "587");
        props.put("mail.smtp.auth", "true");
        props.put("mail.smtp.starttls.enable", "true")
            ;
        System.out.println("SMTP Configuration:");
        System.out.println("  Host: smtp.gmail.com");
        System.out.println("  Port: 587");
        System.out.println("  Auth: true");
        System.out.println("  STARTTLS: true");
        Session session = Session.getInstance(props);
        System.out.println("Session created");
        try {
            MimeMessage message = new MimeMessage(
                session);
            message.setFrom(
                new InternetAddress("sender@test.com")
                    );
            message.setRecipients(
                Message.RecipientType.TO,
                InternetAddress.parse(
                    "receiver@test.com"));
            message.setSubject("Test Mail");
            message.setText(
                "This is a test email sent " +
                "using JavaMail API.");
            System.out.println("\n--- Email Details
                ---");
            System.out.println(
                "From:    sender@test.com");
            System.out.println(
                "To:      receiver@test.com");
            System.out.println(
                "Subject: " + message.getSubject());
            Transport.send(message);
            System.out.println("Email sent
                successfully!");
        } catch (MessagingException e) {
            System.out.println("Error: " +
                e.getMessage());
        }
        System.out.println(
            "\nLab No.: 24\nName: Samir Paudel");
        System.out.println(
            "Roll No./Section: 114-079/D");
    }
}
```

Output

```
[sam@arch labs]$ javac -cp javax.mail.jar:javax.activation.jar Lab24_SendEmail.java && java -cp .:javax.mail.jar:javax.activation.jar Lab24_SendEmail
=== JavaMail Email Sender ===

SMTP Configuration:
  Host: smtp.gmail.com
  Port: 587
  Authentication: true
  STARTTLS: true

✓ Session object created (assume properly configured)

--- Email Details ---
From:    sender@test.com
To:      receiver@test.com
Subject: Test Mail
Body:    This is a test email sent using JavaMail API.

--- Status ---
✓ MimeMessage created successfully
✓ Ready to send (Transport.send() not invoked as per spec)

============================
Lab No.: 24
Name: Samir Paudel
Roll No./Section: 114-079/D
============================
[sam@arch labs]$
```